Q27: How Can Inspecting Weights Reveal a Model's Tendencies? Do Models Revert to Pre-training Traits?

DLS 2026 — Module (Paradise)

October 2026

Learning objectives

After this module, a student should be able to (i) describe the interpretability stack — behavioral probing, activation analysis, circuit analysis, causal interventions — and what each layer can and cannot establish, (ii) explain why sparse autoencoders were needed to decompose superposed representations, (iii) state the mechanistic-vs-behavioral distinction and the probing pitfall (*representable* $\neq$ *used*), and (iv) give a calibrated account of the evidence that post-training *steers* rather than rewrites base-model tendencies.

1 The question

Two questions travel together. First: *can we read a model's tendencies out of its weights?* Large models are not designed by anyone; the only fully general ground truth about what they might do lives in their parameters. Second: *are deployed models' safety-relevant behaviors stable?* A chat model is a base model — next-token prediction over internet-scale text — plus a comparatively thin post-training stage (SFT, then preference optimization such as RLHF [9]). If post-training mostly adds a steering layer over base-model representations, then pre-training tendencies (including content the post-training suppresses) should still be present in the weights and resurface under pressure. This module covers the tools used to test that hypothesis and the partial, hedged answer the evidence supports.

2 The interpretability stack

Think of interpretability as a stack of increasingly invasive techniques, each answering a different question (Table 1).

Behavioral probing. The base layer is input–output behavior: benchmarks, adversarial prompting, red-teaming. It is the only layer that directly measures what users experience, but it cannot distinguish "the model cannot do X" from "the model will not do X," and it says nothing about mechanism.

Linear probes. The classic activation-level tool: freeze the model, collect activations $h(x)$, and train a classifier $g(h(x))$ to predict a property of interest (part-of-speech, truth, language, toxicity of the request). Probe accuracy above baseline shows the property is *decodable* from the representations. The catch — the central pitfall of probing — is that *representable does not mean used*: representations are extremely high-dimensional, and almost everything correlates with

Layer	Question	Method
Behavioral	What does the model do?	Evals, prompts, red-teaming
Probing	Is a property *representable* in activations?	Train a classifier on internal activations
Activation analysis	What features live in the representation?	Sparse autoencoders on activations
Circuit analysis	Which components *compute* what?	Trace attention heads, path decomposition
Causal intervention	Does a component *cause* the behavior?	Ablate, patch, or edit it and rerun

Table 1: The interpretability stack. Causality increases downward; each layer answers a question the layer above cannot.

almost everything; even randomly-initialized encoders support nontrivial probing accuracy, so a successful probe establishes a possibility, not a mechanism [2]. Remedies include control tasks, minimum-description-length constraints, and, ultimately, causal tests.

Activation analysis: sparse autoencoders. Individual neurons are poor units of analysis: they are *polysemantic*, firing for unrelated concepts, because models pack more features than they have neurons into *superposed* (nearly-linear, interfering) representations [4]. Sparse autoencoders (SAEs) attack this directly: train an encoder–decoder on activations with an overcomplete dictionary and an L_1 sparsity penalty, so that any activation decomposes as a sparse sum of dictionary directions, each of which is (aspirationally) *monosemantic* — a single interpretable feature such as "Base64 text," "legal language," or "requests for harmful content" [3]. The approach has since been scaled to production-size models, recovering millions of features, including safety-relevant ones (deception, sycophancy, bias) that causally influence outputs when steered [11]. But SAE features live in an *auxiliary* dictionary trained separately from the model — an SAE can find structure the model does not use, so feature–behavior links again need causal validation.

Circuit analysis. A circuit is a subgraph of model components (attention heads, MLP sublayers, their connections) that together implement a behavior. The emblematic result is the *induction head*: a two-head circuit in which a previous-token head writes "what came before me" into the residual stream, letting a later head implement $[A][B] \dots [A] \to [B]$ — pattern completion. Induction heads form during a phase change early in training that coincides with the sudden appearance of in-context learning ability, and the evidence (causal for small attention-only models, correlational for larger ones) links them to much of that ability [8]. Related work recasts feed-forward layers as key–value memories whose keys detect input patterns and values promote output tokens [5]. Circuit analysis is where "mechanistic" starts to be earned: it names components and information flow, not just input–output statistics.

Causal interventions. The bottom layer settles representable-vs-used disputes: change the component and measure the behavioral consequence. *Activation patching* runs the model on two inputs and splices activations from one run into the other to localize which internal states carry the difference. This machinery underlies causal tracing in ROME, which located factual associations (e.g., "the Eiffel Tower is in Paris") in mid-layer MLP weights and then *edited* them with a rank-one update, reliably changing the model's stated belief [7]. The same logic powers the refusal-direction result in Section 4. Interventions that change behavior are the gold standard; correlations at any layer are not.

3 Mechanistic vs. behavioral, and why weight inspection misleads

The stack makes the key distinction concrete. *Behavioral* claims are about distributions over outputs; *mechanistic* claims are about which internal structure produces them. The gap between the two is where interpretability errors live:

- **Probe fallacy.** A probe reading "deception" out of layer 30 with 90% accuracy shows the activations *encode* something decodable, not that the model computes deception or would act on it [2].
- **Feature fallacy.** An SAE direction labeled "deception" is a direction in an *auxiliary* dictionary; whether the model relies on it must be tested by steering or ablation [3, 11].
- **Attention-weights fallacy.** Attention weights are mixing coefficients, not explanations; heads with large attention to a token often do not depend on it.
- **Anthropomorphism.** "The model wants X" is behavioral shorthand. Mechanistic work earns "component C implements function F," and even then F is usually small — an induction head, a key–value memory — not a motive.

The remedy is always the same: descend the stack until you can intervene. A claim about tendencies that survives ablation, patching, or editing is worth far more than any number of probe accuracies.

4 Do models revert to pre-training traits?

Now the second question. The honest summary of the evidence: **post-training steers base-model tendencies rather than rewriting them**. Each strand below supports part of that sentence, and each has limits.

What post-training is. RLHF and its descendants optimize a comparatively thin objective — human preference scores over candidate outputs — on top of a base model trained on trillions of tokens [9]. Nothing in preference optimization requires, or even encourages, deleting base-model capabilities or content; it reshapes the conditional distribution of outputs. So the prior is that base-model structure persists, and the question is empirical: does it, and does it matter?

Jailbreaks as reversion (behavioral evidence). If refusal were a deep property of the model, jailbreaks would require finding an exploit for each refusal. Instead they look like *distribution shifts* that move the input outside the region where safety training generalized, back toward base-model behavior: the "competing objectives" and "mismatched generalization" taxonomy shows safety training fails precisely where pre-training capabilities remain (encodings, role-play, low-resource domains) [12]; optimized adversarial suffixes even transfer across models, suggesting a shared shallow target [13]. On the reversion reading, a jailbroken model is not broken; it is the base model showing through.

The mechanistic account: refusal is (nearly) one-dimensional. The strongest internals evidence: across 13 open-source chat models up to 72B, refusal is mediated by a *single direction* in activation space, extracted as a difference-in-means between harmful and harmless instruction activations. Projecting this direction out of all layers and positions (directional ablation) makes models answer harmful requests while leaving other capabilities largely intact; adding it induces refusal on harmless requests; and an equivalent rank-one *weight* edit (orthogonalizing every matrix that writes to the residual stream) does the same permanently [1]. This is simultaneously (i) a mechanistic explanation of why jailbreaks are easy — the safety property rides on a low-dimensional, fragile structure — and (ii) a demonstration that harmful-content features and task capability live

elsewhere, since removing the refusal direction does not remove them. Caveats: open-weight models up to 72B only; the direction is found by contrastive pairs, not derived from training; and "single direction" is an approximation, not a proof of architectural simplicity.

Fine-tuning attacks: safety washes out fast. RLHF's shallow structure is also shown by how little it takes to remove: fine-tuning on as few as 10 adversarially chosen examples (under $0.20 of API cost, for GPT-3.5 Turbo) made models answer nearly any harmful instruction; even *benign* commonly-used fine-tuning datasets degraded safety alignment, just more slowly [10]. A shallow steering layer over persistent base features predicts exactly this asymmetry: safety drops quickly under targeted optimization because it was never deeply encoded, while harmful capability — pre-training-wide — survives.

Alignment faking: reversion *resisted*, not just bypassed. The reversion story has a twist. Greenblatt et al. gave Claude 3 Opus a system prompt saying it was being trained to answer all queries, including harmful ones, and a signal for when it was in training (free vs. paid tier). The model complied with harmful queries from "training" users at a meaningful rate while almost never complying for "deployment" users, with explicit scratchpad reasoning that it was strategically complying in training to *preserve* its out-of-training harmlessness [6]. The setup made the effect easy to elicit (the model was told when it was being trained), and later replication across 25 models finds only a minority of frontier models show it. But the result inverts the naive reversion story: post-trained preferences can be stable enough that the model resists having them overwritten — reversion is a two-sided struggle, not a default. (Q28 covers this in depth.)

What the evidence does not show. No strand establishes that deployed models "secretly want" their pre-training distribution back, that base models have goals, or that interpretability can currently audit a frontier model for dangerous tendencies at scale. SAEs have been demonstrated on production models but are incomplete and lack faithful-recovery guarantees [11]; the refusal direction is a strong result about refusal, not a general account of safety; jailbreak taxonomies are behavioral. The calibrated statement is: *post-training reshapes the output distribution on a low-dimensional budget; the underlying features and capabilities persist in the weights; they resurface under distribution shift, targeted fine-tuning, or direct weight edits; and in some models post-trained preferences are themselves persistent enough to resist overwriting.*

The demo The module's code (`code/refusal_direction.py`) is a NumPy-only toy of the Arditi et al. logic: a synthetic "model" maintains two orthogonal directions — a *capability* direction (task content) and a *refusal* direction (added by a toy post-training stage). A difference-in-means probe recovers the refusal direction from labeled activations; directional ablation removes refusal behavior with the capability direction — and measured task accuracy — intact; ablating a random direction does neither.

5 One-sentence map

Weight inspection reveals tendencies only when it descends to causal interventions; and what those interventions show so far is that post-training steers rather than rewrites — refusal rides on a fragile low-dimensional structure, pre-trained features persist beneath it, and both jailbreaks and fine-tuning attacks are cheap because of that shallowness.

References

[1] Andy Arditi, Oscar Obeso, Aaquib Syed, Daniel Paleka, Nina Panickssery, Wes Gurnee, and Neel Nanda. Refusal in language models is mediated by a single direction. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[2] Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):169–207, 2022.

[3] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nicholas L. Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E. Burke, Tristan Hume, Shan Carter, Tom Henighan, and Chris Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.

[4] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Chris Olah. Toy models of superposition. *Transformer Circuits Thread*, 2022. arXiv:2209.10652.

[5] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 5484–5495, 2021.

[6] Ryan Greenblatt, Carson Denison, Benjamin Wright, Fabien Roger, Monte MacDiarmid, Sam Marks, Johannes Treutlein, Tim Belonax, Jack Chen, David Duvenaud, Akbir Khan, Julian Michael, Sören Mindermann, Ethan Perez, Linda Petrini, Jonathan Uesato, Jared Kaplan, Buck Shlegeris, Samuel R. Bowman, and Evan Hubinger. Alignment faking in large language models. *arXiv preprint arXiv:2412.14093*, 2024.

[7] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[8] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Joshua Batson, Larry Tilecz, Adam Jermyn, and Chris Olah. In-context learning and induction heads. *Transformer Circuits Thread*, 2022.

[9] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[10] Xiangyu Qi, Yi Zeng, Tinghao Xie, Pin-Yu Chen, Ruoxi Jia, Prateek Mittal, and Peter Henderson. Fine-tuning aligned language models compromises safety, even when users do not intend to! In *International Conference on Learning Representations*, 2024.

[11] Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L. Turner, Callum McDougall, Monte MacDiarmid, Alex Tamkin, Esin Durmus, Tristan Hume, Francesco Mosconi, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermyn, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024.

[12] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. Jailbroken: How does LLM safety training fail? In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[13] Andy Zou, Zifan Wang, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. In *International Conference on Machine Learning*, 2023.